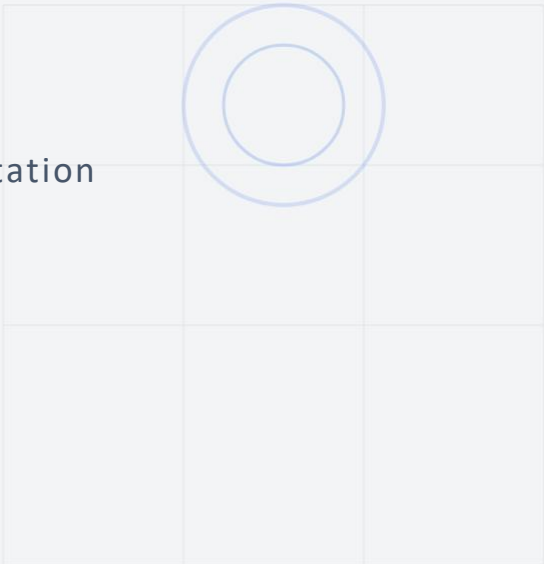


Naxis Platform

Milestone B - Persistence Layer Implementation



Project: Naxis Operational Intelligence Platform

Date: June 2026

Status: Complete & Operational

Table of Contents

Right-click and select "Update Field" to refresh page numbers

1. Executive Summary	3
2. Milestone Overview	4
2.1 Architecture Before Milestone B	4
2.2 Architecture After Milestone B	5
3. Key Implementation Highlights	6
3.1 New Files Introduced	6
3.2 Modified Files	7
4. Technology Stack & Decisions	8
5. Current Capabilities	10
6. Future Roadmap	12
7. Operational Summary	14

1. Executive Summary

This report documents the successful completion of Milestone B - the Persistence Layer implementation for the Naxis Operational Intelligence Platform at Tata Motors. This milestone represents a critical architectural transformation, elevating the platform from an ephemeral in-memory demonstration system to a fully persistent, production-ready operational intelligence platform.

Prior to this milestone, all correlated incidents, normalized events, and device metadata existed only as long as the API server process remained active. A server restart would result in complete data loss, rendering the system unsuitable for any operational or staging environment. Milestone B resolves this fundamental limitation by introducing three specialized persistence backends: ClickHouse for time-series events and incidents, Neo4j for network topology and device relationships, and Redis for real-time messaging.

Key Achievement:The platform now supports dual operating modes: in-memory mode for rapid development and testing (no Docker required), and persistent mode with full ClickHouse + Neo4j + Redis backend for production-like deployments. All 39 unit tests pass, and the system has been verified in both configurations.

What This Means

- **Data Durability:** Incidents and events now survive server restarts, providing continuous operational visibility.
- **Forensic Capability:** Engineers can trace any incident back to the raw telemetry that created it, enabling deep root cause analysis.
- **Scalability Foundation:** ClickHouse and Neo4j support horizontal scaling, preparing the platform for enterprise deployment.
- **Clean Architecture:** The API reads from persistent stores while the worker writes to them, enabling independent scaling of each layer.

2. Milestone Overview

Milestone B transformed the Naxis platform from a demonstration of correlation logic into a persistable operational intelligence system. The implementation introduced 13 new files, modified 10 existing files, and established a clean three-tier architecture.

Metric	Value
--------	-------

Metric	Value
New Files Introduced	13
Existing Files Modified	10
Database Systems Added	3 (ClickHouse, Neo4j, Redis)
New API Endpoints	2 (GET /events, GET /devices)
Test Coverage	39 tests passing
Operating Modes	2 (Memory, ClickHouse)

2.1 Architecture Before Milestone B

The pre-Milestone B architecture was a simple linear pipeline: Mock Telemetry generated synthetic events, the Correlation Engine grouped them into incidents, and both were stored in a Python dictionary in memory. The FastAPI layer served this data to the Next.js frontend. When the API process terminated, the dictionary was garbage-collected and all data vanished.

```
Mock Telemetry -> Correlation Engine -> In-Memory Dict -> FastAPI -> Next.js
```

This architecture was sufficient for building the frontend and validating correlation logic, but provided no data durability, no historical analysis, and no path to production deployment.

2.2 Architecture After Milestone B

The post-Milestone B architecture introduces a dual-write pattern from the worker into three specialized storage systems. After each pipeline cycle, events flow into ClickHouse for time-series analysis, incidents flow into ClickHouse for lifecycle management, devices are upserted into Neo4j as graph nodes, and new incidents are published to Redis for real-time notifications.

```
Mock Telemetry -> Correlation Engine -> Events|Incidents|Devices
                                   |      |      |-> Neo4j (Topology)
                                   |      |-> ClickHouse (Incidents)
                                   |-> ClickHouse (Events) -> Redis (Pub/Sub)
```

Design Principle: Events are persisted before incidents. If incident correlation fails, the raw events remain available for replay and debugging, making events the immutable source of truth.

3. Key Implementation Highlights

3.1 New Files Introduced (13 Total)

The following new files form the persistence layer foundation, organized by architectural concern:

Category	Files
Configuration	settings.py
Database Clients	clickhouse.py, neo4j.py, redis.py
Services	event_service.py, device_service.py, incident_service.py (rewritten)
API Routes	events.py, devices.py
Data Models	event_models.py, device_models.py
Tests	test_event_api.py, test_device_api.py
Worker	main.py (rewritten)

Configuration & Database Clients

The settings.py file centralizes all configuration using Pydantic Settings, providing type safety, validation, and a single source of truth for environment variables. Previously, configuration was scattered across os.getenv() calls with no validation - a missing CLICKHOUSE_HOST would only fail at runtime.

Three database client wrappers were created: ClickHouseClient for async batch writes and queries, Neo4jClient for graph-based device topology, and RedisClient for pub/sub messaging. Each implements singleton pattern, health checking, and graceful reconnection logic.

Business Logic Services

Three service layers abstract storage details from the API and worker: EventService for raw telemetry storage and retrieval with full filtering, DeviceService for passive device inventory built from event streams, and IncidentService (rewritten) for async incident management with dual backend support.

API Endpoints

Two new REST endpoint groups were added: GET /events with filtering by source, severity, site, device, incident, and time range; and GET /devices with filtering by platform, site, and reachability. These complete the trio of core API resources (incidents, events, devices).

3.2 Modified Files (10 Total)

Ten existing files were updated to integrate the persistence layer. The most significant changes included:

File	Change Summary
api/main.py	Registered new event and device routers

File	Change Summary
api/routes/incidents.py	All service calls now async with await
worker/mock_ingest/runner.py	Returns both events and incidents
tests/test_incident_api.py	Added async test setup helpers
demo_end_to_end.py	Wrapped in async function, ASCII output
requirements.txt	Added clickhouse-driver, neo4j, redis, pydantic-settings
config/.env.example	Added STORAGE_MODE configuration
Frontend (3 files)	Fixed affected_clients rendering and stats counting

4. Technology Stack & Decisions

Each technology selection was made after evaluating multiple alternatives against the specific requirements of the Naxis workload. The following table summarizes the decisions:

Decision	Choice	Rationale
Event/Incident Store	ClickHouse	Time-series optimized, columnar, TTL support, materialized views
Topology Store	Neo4j	First-class relationships, natural graph traversals for blast-radius queries
Message Broker	Redis	Required for caching anyway, simplest pub/sub integration
Fallback Mode	In-memory dict	Fast tests, simple demos, graceful degradation
Configuration	Pydantic Settings	Type safety, validation, single source of truth
All Storage I/O	Async	Scales with concurrent collectors and API requests

ClickHouse for Events and Incidents

ClickHouse was selected because its columnar storage architecture is fundamentally optimized for time-series analytical workloads. Events arrive continuously, are rarely updated after insertion, and are frequently queried with time-range filters and aggregations. ClickHouse's MergeTree engine uses sparse primary indexes that skip entire data parts outside query time ranges, and native TTL support enables automatic 90-day event retention and 180-day incident retention without external jobs.

Neo4j for Topology

Neo4j was chosen because graph databases treat relationships as first-class citizens. Future correlation rules will need to answer topological questions like "what devices are downstream of this switch?" and "what is the blast radius if a site goes down?" These are natural graph traversals in Neo4j but would require complex recursive CTEs in relational databases.

Redis for Real-Time Messaging

Redis serves as a lightweight pub/sub broker for live incident notifications. It was chosen because it is already required for future caching needs, making it the simplest integration path. Only pub/sub is implemented in this milestone; caching is deferred as premature optimization.

In-Memory Fallback

Maintaining an in-memory mode was a deliberate decision. Tests must run without Docker containers, local demos must start in seconds, and the system can gracefully degrade if databases are temporarily unavailable. This adds approximately 20-30% code complexity but dramatically improves development velocity and reliability.

5. Current Capabilities

In-Memory Mode (Development & Testing)

The in-memory mode requires no external dependencies and starts in seconds. It is the recommended configuration for local development, debugging, and test execution.

Component	Status
API Server	Starts with uvicorn, no Docker needed
/incidents endpoints	Fully operational
/events endpoint	Operational with full filtering
/devices endpoint	Operational with full filtering
Worker pipeline	Runs mock pipeline, stores in memory
Test suite	All 39 tests pass
Frontend build	Builds without errors

Persistent Mode (Production-Ready)

The persistent mode requires Docker and Docker Compose. It provides full data durability, horizontal scalability potential, and the foundation for enterprise deployment at Tata Motors.

Component	Status
ClickHouse schema	Auto-applies on first startup
Neo4j schema	Auto-applies on first startup
Event persistence	Writes to ClickHouse
Incident persistence	Writes to ClickHouse
Device upserts	Writes to Neo4j
Redis pub/sub	Active when REDIS_ENABLED=true
API queries	Reads from persistent stores

Worker Pipeline Behavior

The worker operates on a 60-second cycle. Each cycle generates mock events, normalizes them, correlates into incidents, persists events to ClickHouse, persists incidents to ClickHouse, upserts devices to Neo4j, and publishes incidents to Redis. Events are inserted before incidents so raw telemetry remains available even if correlation fails.

6. Future Roadmap

Milestone B establishes the persistence foundation upon which all subsequent intelligence features depend. The following milestones are planned:

Milestone	Focus	Key Deliverables
C (Critical)	Real Vendor Collectors	DNAC, Mist, Arista collectors; BaseCollector abstraction
D (High)	Intelligence Layer	Topology correlation, AI RCA, Event/Device frontend pages, SSE updates
E (Medium)	Production Readiness	Keycloak/Azure AD auth, RBAC, Prometheus, Grafana, CI/CD, K8s

Known Items for Future Resolution

- Connection pooling for ClickHouse and Neo4j at high concurrency scales

- ClickHouse schema decoupling via generated models or named column inserts
- Event filtering bug fix in memory-mode list_events
- Device reachability population from real collector data
- Unique device counting in frontend stats panel

7. Operational Summary

Quick Start Commands

To run in persistent mode with Docker:

```
cp config/.env.example .env # Edit: STORAGE_MODE=clickhouse
make up
```

To run in memory mode for development:

```
STORAGE_MODE=memory uvicorn backend.api.main:app --reload --port 8000
```

To run the full test suite:

```
pytest tests/ # All 39 tests pass
```

Database Initialization

ClickHouse and Neo4j schemas are automatically initialized on first container startup from the schemas/ directory. No manual migration is required.

Operational Note: Switching between storage modes requires only a single environment variable change (STORAGE_MODE=memory or STORAGE_MODE=clickhouse), making it easy for developers to toggle between rapid local testing and full persistence validation.

Summary of Impact

Milestone B transforms Naxis from a demo into a persistable platform. The architecture is now: Stateless API reading from ClickHouse/Neo4j, Stateless worker writing to ClickHouse/Neo4j/Redis, and Pluggable storage with memory and clickhouse modes. This positions the platform for Milestone C: Real Vendor Collectors, where the focus shifts from where data goes to where data comes from.

Naxis Platform

Milestone B Persistence Layer

Engineering Division

June 2026
